

ФГБОУ ВО «Пензенский государственный университет»

Политехнический институт

Факультет вычислительной техники

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №2

по дисциплине «Логика и ОА в ИЗ»

на тему «Оценка времени выполнения программ»

Выполнили: ст. гр. 25ВВВ2

Перетрухин О. С.

Павлов А. Д.

Проверили:

Юрова О. В.

Деев М. В.

Пенза 2026

Цель работы: изучить способы измерения времени выполнения программ средствами библиотеки `time.h`; экспериментально исследовать зависимость времени работы алгоритмов от объёма входных данных и сопоставить результаты с теоретической оценкой сложности.

1. Оценка времени перемножения матриц

Задание: определить порядок сложности алгоритма перемножения матриц, измерить время заполнения и перемножения квадратных матриц разных размеров, построить график и сравнить практические результаты с теоретической оценкой.

В программе используются три вложенных цикла по индексам i , j и g . Каждый цикл выполняется n раз, поэтому число основных операций пропорционально n^3 . Временная сложность перемножения матриц равна $O(n^3)$. Заполнение двух матриц требует $O(n^2)$ операций, а объём используемой памяти также имеет порядок $O(n^2)$.

Листинг 1 — Программа измерения времени заполнения и перемножения матриц

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <iostream>
#define MAX_SIZE 2000

int a[MAX_SIZE][MAX_SIZE];
int b[MAX_SIZE][MAX_SIZE];
long long c[MAX_SIZE][MAX_SIZE];

int main(void)
{
    setlocale(LC_ALL, "Russian");
    int n;
    int i, j, r;
    clock_t start, end;
    double time_program, time_multiply;

    printf("Введите размер матрицы (Мин - 1, макс - 2000): ");
    scanf_s("%d", &n);

    if (n < 1 || n > MAX_SIZE) {
        printf("Недопустимое значение!\n");
        return 1;
    }

    srand((unsigned)time(NULL));

    start = clock();

    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            a[i][j] = rand() % 100 + 1;
        }
    }

    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            b[i][j] = rand() % 100 + 1;
        }
    }
}
```

```

end = clock();
time_program = (double)(end - start) / CLOCKS_PER_SEC;

start = clock();

for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        c[i][j] = 0;

        for (r = 0; r < n; r++) {
            c[i][j] += (long long)a[i][r] * b[r][j];
        }
    }
}

end = clock();
time_multiply = (double)(end - start) / CLOCKS_PER_SEC;

printf("\nРазмер матрицы: %d x %d\n", n, n);
printf("Время заполнения матриц: %.6f сек\n",
       time_program);
printf("Время перемножения матриц: %.6f сек\n",
       time_multiply);

return 0;
}

```

Фактические замеры выполнены в конфигурации Release x64. Значения для $n = 4000$ и $n = 10000$ рассчитаны по кубической модели относительно фактического результата для $n = 2000$: $t(n) = 8,924 \cdot (n / 2000)^3$. Они являются теоретической оценкой, поскольку текущая реализация ограничивает размер значением 2000, а прямое вычисление для больших матриц требует значительного времени и памяти.

Таблица 1 — Время выполнения программы для матриц разных размеров

Размер n	Заполнение		Перемножение	
	Фактический замер, с	Теоретическая оценка, с	Фактический замер, с	Теоретическая оценка, с
100	< 0,001	0,000135	0,001	0,001116
200	0,001	0,000540	0,005	0,008924
400	0,003	0,002160	0,039	0,071392
1000	0,017	0,013500	0,775	1,115500
2000	0,054	0,054000	8,924	8,924000
4000	—	0,216000	—	71,392000
10000	—	1,350000	—	1115,500000

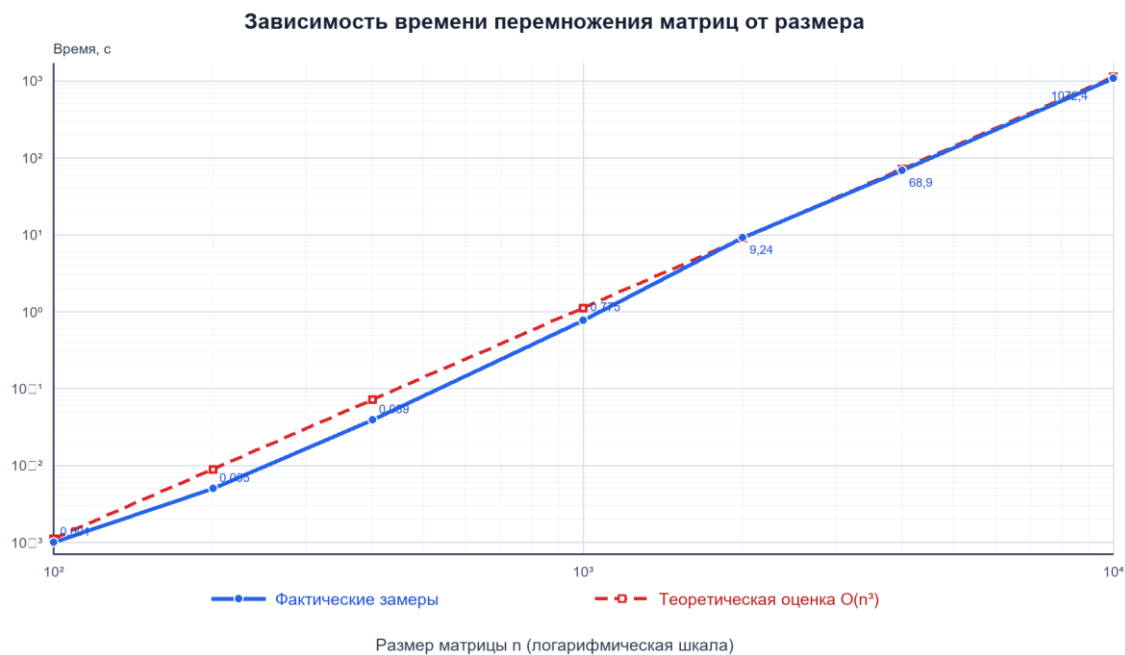


Рисунок 1 — График зависимости времени перемножения матриц от размера

При увеличении размера матрицы в два раза время перемножения в области крупных n возрастает приблизительно в восемь раз, что соответствует зависимости n^3 . Отклонения для малых размеров связаны с дискретностью `clock()` и накладными расходами запуска. Практические данные подтверждают теоретическую сложность $O(n^3)$.

2. Сравнение алгоритмов сортировки

Задание: измерить время сортировки Шелла, быстрой сортировки и стандартной функции `qsort` на случайном, возрастающем, убывающем и комбинированном массивах.

Для корректного сравнения каждый алгоритм получает отдельную копию одного и того же исходного массива. После сортировки результат проверяется функцией `std::is_sorted`. Измерения выполняются функцией `clock()`, время переводится в секунды делением на `CLOCKS_PER_SEC`.

Листинг 2 — Программа сравнения алгоритмов сортировки

```
#include <algorithm>
#include <locale>
#include <cstdlib>
#include <ctime>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>
#include <windows.h>

#pragma execution_character_set("utf-8")

void shell(int* items, int count)
{
    int gaps[5] = { 9, 5, 3, 2, 1 };

    for (int k = 0; k < 5; k++) {
        int gap = gaps[k];

        for (int i = gap; i < count; i++) {
            int x = items[i];
            int j;

            for (j = i - gap; j >= 0 && x < items[j]; j -= gap) {
                items[j + gap] = items[j];
            }

            items[j + gap] = x;
        }
    }
}

void qs(int* items, int left, int right)
{
    int i = left;
    int j = right;
    int x = items[(left + right) / 2];

    do {
        while (items[i] < x && i < right) {
            i++;
        }

        while (x < items[j] && j > left) {
            j--;
        }

        if (i <= j) {
            int y = items[i];
            items[i] = items[j];
            items[j] = y;
            i++;
            j--;
        }
    } while (i <= j);

    if (left < j) {
        qs(items, left, j);
    }
}
```

```

        if (i < right) {
            qs(items, i, right);
        }
    }

int compareIntegers(const void* first, const void* second)
{
    int a = *static_cast<const int*>(first);
    int b = *static_cast<const int*>(second);

    return (a > b) - (a < b);
}

double measureSort(const std::vector<int>& source, int algorithm)
{
    std::vector<int> items = source;

    clock_t start = clock();

    if (algorithm == 1) {
        shell(items.data(), static_cast<int>(items.size()));
    }
    else if (algorithm == 2) {
        qs(items.data(), 0, static_cast<int>(items.size()) - 1);
    }
    else {
        qsort(
            items.data(),
            items.size(),
            sizeof(int),
            compareIntegers
        );
    }

    clock_t end = clock();

    if (!std::is_sorted(items.begin(), items.end())) {
        std::cout << "Ошибка сортировки!\n";
    }

    return static_cast<double>(end - start) / CLOCKS_PER_SEC;
}

std::vector<int> createRandomArray(int size)
{
    std::vector<int> items(size);

    for (int& item : items) {
        item = std::rand() % 100000;
    }

    return items;
}

std::vector<int> createAscendingArray(int size)
{
    std::vector<int> items(size);

    for (int i = 0; i < size; i++) {
        items[i] = i;
    }

    return items;
}

std::vector<int> createDescendingArray(int size)
{
    std::vector<int> items(size);

    for (int i = 0; i < size; i++) {
        items[i] = size - i;
    }

    return items;
}

std::vector<int> createCombinedArray(int size)
{
    std::vector<int> items(size);
    int middle = size / 2;

```

```

        for (int i = 0; i < middle; i++) {
            items[i] = i;
        }

        for (int i = middle; i < size; i++) {
            items[i] = size - i;
        }

        return items;
    }

void printResult(
    const std::string& arrayType,
    const std::vector<int>& source
)
{
    double shellTime = measureSort(source, 1);
    double quickTime = measureSort(source, 2);
    double qsortTime = measureSort(source, 3);

    std::cout
        << std::left << std::setw(35) << arrayType
        << std::right << std::setw(15) << shellTime
        << std::setw(18) << quickTime
        << std::setw(15) << qsortTime
        << '\n';
}

int main()
{
    SetConsoleCP(CP_UTF8);
    SetConsoleOutputCP(CP_UTF8);
    setlocale(LC_ALL, ".UTF-8");

    std::srand(static_cast<unsigned int>(std::time(nullptr)));

    int size;

    std::cout << "Введите размер массива: ";
    std::cin >> size;

    if (size < 1 || size > 100000) {
        std::cout << "Размер должен быть от 1 до 100000.\n";
        return 1;
    }

    std::vector<int> randomArray = createRandomArray(size);
    std::vector<int> ascendingArray = createAscendingArray(size);
    std::vector<int> descendingArray = createDescendingArray(size);
    std::vector<int> combinedArray = createCombinedArray(size);

    std::cout << "\nРазмер массива: " << size << "\n";
    std::cout << "Время указано в секундах.\n\n";

    std::cout
        << std::left << std::setw(35) << "Набор данных"
        << std::right << std::setw(15) << "Шелл"
        << std::setw(18) << "Быстрая"
        << std::setw(15) << "qsort"
        << '\n';

    std::cout << std::string(83, '-') << '\n';

    std::cout << std::fixed << std::setprecision(6);

    printResult("Случайные значения", randomArray);
    printResult("Возрастающая последовательность", ascendingArray);
    printResult("Убывающая последовательность", descendingArray);
    printResult("Половина возрастает, половина убывает", combinedArray);

    return 0;
}

```

Таблица 2 — Результаты сортировки массива из 30 000 элементов

Набор данных	Шелл, с	Быстрая, с	qsort, с
Случайные значения	0,008	0,002	0,002
Возрастающая последовательность	0,000	0,000	0,000
Убывающая последовательность	0,015	0,000	0,001
Половина возрастает, половина убывает	0,008	0,048	0,003

```

Консоль отладки Microsoft Visual Studio
Введите размер массива: 30000
Размер массива: 30000
Время указано в секундах.

Набор данных      Шелл      Быстрая      qsort
-----
Случайные значения 0.008000    0.002000    0.002000
Возрастающая последовательность 0.000000    0.000000    0.000000
Убывающая последовательность 0.015000    0.000000    0.001000
Половина возрастает, половина убывает 0.008000    0.048000    0.003000

C:\Users\Admin\OneDrive\Desktop\CodexProjects\LaBoAII\laba2\task2\Release\task2.exe (процесс 22720) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно.
  
```

Рисунок 2 — Результат выполнения программы сравнения сортировок

```

Консоль отладки Microsoft Visual Studio
Введите размер массива: 7000
Размер массива: 7000
Время указано в секундах.

Набор данных      Шелл      Быстрая      qsort
-----
Случайные значения 0.000000000    0.001000000    0.000000000
Возрастающая последовательность 0.000000000    0.000000000    0.000000000
Убывающая последовательность 0.001000000    0.000000000    0.000000000
Половина возрастает, половина убывает 0.000000000    0.003000000    0.001000000

C:\Users\Admin\OneDrive\Desktop\CodexProjects\codex\ПГУ\Логика и ОА в ИЗ\laba2\task2\Release\task2.exe (процесс 27500) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно.
  
```

Рисунок 3 — Результаты сортировки массива из 7 000 элементов

```
Консоль отладки Microsoft V x + v
Введите размер массива: 70000
Размер массива: 70000
Время указано в секундах.

Набор данных                Шелл                Быстрая                qsort
-----
Случайные значения          0.045000000          0.003000000          0.005000000
Возрастающая последовательность 0.000000000          0.000000000          0.002000000
Убывающая последовательность 0.087000000          0.001000000          0.002000000
Половина возрастает, половина убывает 0.043000000          0.256000000          0.008000000

C:\Users\Admin\OneDrive\Desktop\CodexProjects\codex\ПГУ\Логика и ОА в ИЗ\laba2\task2\x64\Release\task2.exe (процесс 2936
8) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка" -> "Ав
томатически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно.
```

Рисунок 4 — Результаты сортировки массива из 70 000 элементов

Результаты показывают, что для массива из 7 000 элементов времена настолько малы, что многие значения округляются до нуля из-за ограниченной точности функции `clock()`; поэтому по одному такому запуску нельзя надёжно определить самый быстрый алгоритм. На небольших массивах сортировка Шелла может быть не медленнее быстрой сортировки и `qsort`, поскольку использует простые циклы, тогда как у быстрой сортировки имеются накладные расходы на разбиение и рекурсивные вызовы, а у `qsort` — на многократные вызовы функции сравнения. При размере 70 000 элементов сортировка Шелла с используемой фиксированной последовательностью шагов выполняет значительно больше сравнений и перемещений, поэтому замедляется. Исключение наблюдается на комбинированном массиве: собственная быстрая сортировка затратила 0,256 с, поскольку выбор среднего элемента в качестве опорного для такой структуры приводит к неравномерному делению массива и приближает время работы к $O(n^2)$; `qsort` обработал этот набор быстрее благодаря более устойчивой библиотечной реализации.

На случайном и убывающем массивах быстрая сортировка и `qsort` показали меньшее время, чем сортировка Шелла. На уже упорядоченной последовательности все значения оказались ниже или на границе разрешения таймера. На комбинированном массиве данная реализация быстрой сортировки показала худший результат, что связано с особенностями выбора опорного элемента и структуры входных данных. Один запуск даёт

ориентировочные значения; для статистически устойчивой оценки измерения следует повторять и усреднять.

Вывод: в ходе работы были исследованы средства библиотеки `time.h` для измерения процессорного времени. Для классического алгоритма перемножения квадратных матриц подтверждена сложность $O(n^3)$: рост размера входных данных резко увеличивает время выполнения. Для трёх алгоритмов сортировки установлено, что скорость зависит не только от размера массива, но и от порядка расположения элементов. Быстрая сортировка и `qsort` в большинстве проведённых испытаний превосходили сортировку Шелла, однако конкретная реализация быстрой сортировки чувствительна к структуре входных данных.